If you'd like to know how to measure all of the agile tenets, head over to chapter 10 where I've broken them down into what you should measure and how to do it.

8.1. Preparing for analysis: setting up to measure your code

In earlier chapters we looked into the tools and practices you should be following to get the data you need. In chapter 5 we talked about using static analysis in your CI systems, and in chapter 6 we walked through using APM tools to gather data on how well your application is functioning and if it's doing what it's supposed to do. The data generated from components in your CI and your APM systems will get you what you need.

As a reference, here's a list of the tools we've used in previous chapters and that we'll be referencing in this chapter.

Tool	Measures	Metrics
New Relic	Application monitoring	Page response time, uptime, response time, error rate
HyperSpin	Availability	Uptime, response time
Splunk	Reliability	Error rate, mean time between failures
OWASP ZAP	Security	Dynamic analysis issues
SonarQube	Maintainability	CLOC, code coverage, issues, complexity
Checkmarx	Security	Static analysis issues

8.2. Measuring the NFRs through the code "ilities"

The code "ilities," or non-functional requirements[1] (NFRs), are a set of properties that describe how well software is built. These aren't anything new; any software engineer should be familiar with them and how they indicate the quality of software. Here are some examples of non-functional requirements that indicate how well-built software is:

[1] For much more detail, see en.wikipedia.org/wiki/Non-functional_requirement.